

08 · Session 状态、JSONL 树、分支与恢复

消息、工具结果、标签、Compaction 与当前 leaf 如何组成可恢复的树形历史

分析版本：027a5847901b5dde30270abaa1041046cd2b4b55

核心结论

SessionManager 把会话保存为 append-only JSONL；除 header 外每个 SessionEntry 都有 id/parentId，因此文件物理上是线性日志，逻辑上是树。当前 leaf 决定 active branch。恢复时沿 parentId 从 leaf 走到 root，再应用 model/thinking 状态与 compaction checkpoint，把 SessionEntry 投影成 AgentMessage。

1. 为什么“JSONL + parentId”比重写整棵树简单

每次新事件只 append 一行，天然适合崩溃恢复和流式持久化；分支只需把新 entry.parentId 指向历史节点，不必复制旧消息或重写文件。代价是读取 active context 时必须建立 id index 并沿父链重建路径。

2. SessionHeader 与 SessionEntry 的边界

第一行 SessionHeader 含 version、session id、timestamp、cwd、可选 parentSession，不属于树。之后的所有 SessionEntryBase 都含 type/id/parentId/timestamp。当前版本 CURRENT_SESSION_VERSION=3，并有 v1->v2（引入树 id）与 v2->v3（hookMessage -> custom）迁移。

3. 主要 Entry 类型及是否进入 LLM context

Entry	用途	进入 LLM Context
message	user/assistant/toolResult 等 AgentMessage	是
model_change	模型切换事件	不直接；恢复设置
thinking_level_change	推理等级切换	不直接；恢复设置
compaction	旧上下文摘要/checkpoint	是，投影为 compactionSummary
branch_summary	离开分支的摘要	是，投影为 branchSummary
custom	扩展持久化状态	否
custom_message	扩展注入上下文	是，投影为 custom AgentMessage
label	书签/标记目标 entry	否
session_info	会话显示名等元数据	否

4. 树是如何形成的

```
A(user) -> B(assistant) -> C(user) -> D(assistant)
                                     |
                                     +-> E(user alternate) -> F(assistant)

leaf=D  => active path A-B-C-D
leaf=F  => active path A-B-C-E-F
```

buildSessionPath 建立 id->entry map，从指定 leaf（或默认最后 entry）不断跟 parentId，最后 reverse。选择 /tree 节点本质就是改变 leaf，再从该节点继续 append 新 child。

5. /tree 选择 user 与选择 assistant 为什么行为不同

选择 user/custom message 时，UI 把 leaf 移到该消息的 parent，并把被选消息文本放回编辑器；重新提交会创建“编辑后的新分支”。选择 assistant/tool/compaction 等非用户节点则直接把 leaf 移到该 entry，编辑器为空，允许从其结果继续。

6. SessionEntry -> AgentMessage 投影

sessionEntryToContextMessages 是恢复模型的关键边界：message 原样返回；custom_message 变 CustomMessage；branch_summary 变 BranchSummaryMessage；compaction 变 CompactionSummaryMessage；model_change/label/custom/session_info 等返回空。这样持久化格式可以比 LLM transcript 更丰富。

7. 恢复 model 与 thinking level

getSessionContextSettings 顺 active path 扫描 thinking_level_change、model_change，并以 assistant message 的 provider/model 作为模型信息来源之一。恢复时 createAgentSession 先 buildSessionContext，再尝试从 ModelRuntime 找对应模型；找不到才 fallback。

8. Compaction 让“路径重建”变成 checkpoint 重建

buildContextEntries 不会简单返回 root->leaf 全路径。存在 compaction 时，旧的已总结条目被省略，compaction summary + firstKeptEntryId 之后的条目成为新上下文。较新的格式还可持有 retainedTail，使 compaction entry 自身成为更独立的 checkpoint。

9. 标签为什么是独立 entry，而不是修改目标节点

LabelEntry 记录 targetId 和 label，这符合 append-only 日志原则：修改/清除标签也追加新记录，不必原地改旧 JSONL 行。SessionManager 在构建树时解析每个 target 的“最新标签状态”。这同 session_info、model_change 的事件式建模一致。

10. 对象所有权与生命周期

对象	所有者	生命周期/失效
----	-----	---------

SessionManager	AgentSession/Runtime	整个打开会话；switch/new 时替换
entries/byId/tree	SessionManager 内存表示	从 JSONL 加载，append 时增长
leafId	SessionManager	tree navigation 时改变
Agent.state.messages	Agent	从 buildSessionContext 恢复并随 run 增长
Extension captured ctx	Extension runtime	session replacement/reload 后显式标 stale

11. 故障与兼容风险

- JSONL 可包含 malformed 行：parseSessionEntries 会跳过，调试时不能假设每一行都被加载。
- 手工编辑或旧 session 可能有 null content；sessionEntryToContextMessages 做防御性归一化。
- 改变 entry schema 必须维护 migration；否则老会话无法恢复。
- 分支/clone/fork 区别在“同文件树”与“新 session file”，修改相关逻辑时不要混淆 parentId 与 parentSession。
- custom entry 不进 LLM；若扩展希望模型看到内容，必须使用 custom_message/sendMessage。

动态验证方案

实验 1：验证树而非线性列表

假设：跳回旧节点后新消息 parentId 应指向旧节点/其父

操作：创建 3 轮会话，用 /tree 回第一轮再发新消息，检查 JSONL。

预期：文件末尾新增 entry，但 parentId 形成第二个 child，active leaf 指向新分支。

若不一致：若旧内容被重写，说明使用了不同实现/导出格式。

实验 2：验证恢复投影

假设：custom 不进模型，custom_message 会进

操作：扩展分别 appendEntry 与 sendMessage，重启 session 并打印 buildSessionContext.messages。

预期：只有 custom_message 对应 AgentMessage。

若不一致：若 custom 状态进入上下文，检查扩展使用 API 是否混淆。

实验 3：验证版本迁移

假设：v1/v2 session 自动迁到 v3 内存形态

操作：复制旧 fixture，用 migrateSessionEntries 或加载 SessionManager。

预期：得到 id/parentId，hookMessage 改为 custom。

若不一致：若失败，检查 header version 与 malformed JSON。

源码证据索引

文件	关键符号/用途	本报告结论
packages/coding-agent/src/core/session-manager.ts	SessionEntry types / migrate / buildSessionPath / buildContextEntries	核心存储与恢复算法
packages/coding-agent/src/core/messages.ts	Custom/Bash/Branch/Compaction message	Entry 到 AgentMessage 的扩展类型
packages/coding-agent/src/core/agent-session.ts	message_end persistence / session switch	运行状态与持久化连接
packages/coding-agent/docs/session-format.md	JSONL schema / tree / context building	行为文档交叉验证
packages/coding-agent/docs/sessions.md	/tree / fork / clone 语义	用户行为契约

证据状态：除非单独标注，本报告中的行为结论均来自上述固定 commit 的静态源码与仓库测试/文档交叉验证；未实际启动 Pi、未抓取真实网络请求，因此“运行结果已验证”项为空。

推荐阅读顺序

优先级	文件	阅读目的	精读
1	packages/coding-agent/src/core/session-manager.ts	先看 types、buildSessionPath、buildContextEntries	是
2	packages/coding-agent/docs/session-format.md	带着源码核对每种 entry	是
3	packages/coding-agent/src/core/messages.ts	理解持久化消息如何投影给 LLM	是
4	packages/coding-agent/src/core/agent-session.ts	看 message_end 如何 append	否
5	packages/coding-agent/src/modes/interactive/components/tree-selector.ts	最后看 UI 如何操纵 leaf	否